

Guia de Arquitetura: Domínio Dev & Segurança Ofensiva (Sandbox) — Ravena AI v3.3.0 (Conceitual)

Autor: Manus AI

Data: 25 de Abril de 2026

Versão: 3.3.0 (Conceitual)

Introdução

Este guia conceitual apresenta a evolução da arquitetura da Ravena AI para a versão 3.3.0, com foco na expansão do domínio de desenvolvimento (**Agente Dev**) e na introdução de um novo agente especializado em segurança ofensiva (**Agente Hacker**). A principal motivação é garantir a **segurança sistêmica** e a **integridade do código** através de um processo de desenvolvimento e validação robusto, utilizando **Sandboxes isoladas** para prevenir erros sistêmicos e ataques **1**.

1. Definição de Escopo e Responsabilidades dos Agentes

A nova arquitetura redefine e expande os papéis dos agentes envolvidos no ciclo de vida do desenvolvimento e segurança do código da Ravena.

1.1. Agente Dev (O Construtor)

O **Agente Dev** evolui para um papel de **Arquiteto de Soluções e Desenvolvedor**. Sua principal responsabilidade é a criação, manutenção e otimização de código funcional, aderindo estritamente às diretrizes de design e modularidade da Ravena.

- Responsabilidades Primárias:**

- Desenvolvimento de novas funcionalidades e módulos para o **OmegaCore**.
- Refatoração e otimização de algoritmos existentes para performance e eficiência.
- Geração de documentação técnica e testes unitários para o código produzido.
- Proposição de melhorias arquiteturais e tecnológicas.

- Limitação Crítica:** O **Agente Dev** **não possui permissão para executar código diretamente no ambiente de produção** da Ravena. Todo o código deve passar por um processo de validação rigoroso na Sandbox antes de ser integrado ao Core **2**.

1.2. Agente Hacker (O Auditor de Segurança / Red Team)

Este é um novo agente, especializado em **Segurança Ofensiva**. Seu objetivo é atuar como uma **Red Team** interna, buscando ativamente por vulnerabilidades e falhas no código desenvolvido pelo **Agente Dev** antes que ele possa impactar o sistema principal. A filosofia é clara: "Se o Agente Hacker não conseguir quebrar, o código é seguro para o Core" ³.

- **Responsabilidades Primárias:**
 - **Análise Estática de Código (SAST):** Identificação de padrões de código vulneráveis, como injeções de SQL, credenciais *hardcoded* ou falhas de lógica de segurança.
 - **Análise Dinâmica de Aplicações (DAST) / Penetration Testing (Pentest):** Execução de testes de intrusão e exploração de vulnerabilidades em ambientes de teste controlados (Sandboxes).
 - **Auditoria de Dependências:** Verificação de bibliotecas e pacotes de terceiros em busca de vulnerabilidades conhecidas (CVEs).
 - **Simulação de Ataques:** Emulação de cenários de ataque (e.g., DoS, exfiltração de dados) para testar a resiliência dos mecanismos de defesa da Ravena (Circuit Breaker, SecurityLayer, Lockdown).
- **Filosofia Operacional:** Atuar como um adversário constante, garantindo que apenas código robusto e seguro seja promovido para o ambiente de produção.

2. Arquitetura de Sandbox Isolada para Execução Segura

Para garantir que as atividades do **Agente Dev** (testes de código) e do **Agente Hacker** (testes de segurança ofensiva) não comprometam a estabilidade ou a segurança do **OmegaCore**, é fundamental a implementação de uma **Arquitetura de Sandbox Isolada**. Este ambiente de isolamento multinível previne "erros sistêmicos" e a propagação de falhas ⁴.

2.1. Camadas de Isolamento

Camada	Componentes Principais	Acesso e Permissões	Recursos de Rede	Persistência
Core (Produção)	OmegaCore , APIs de Trading (Bybit), OCI Services, Bancos de Dados de Produção, Chaves de API Reais	Somente leitura para Agente Dev . Nenhum acesso direto para Agente Hacker .	Full (Bybit, OCI, Telegram)	Persistente e Criptografado

Sandbox (Dev/Hacker)	Ambiente de execução de código, ferramentas de análise, dados de teste (mocks)	Agente Dev e Agente Hacker com privilégios limitados (sandbox user, sem root).	Restrito (Apenas repositórios de pacotes, sem acesso ao Core)	Volátil (Destruído após o uso)
-----------------------------	--	---	---	--------------------------------

2.2. Fluxo de Execução Seguro na Sandbox

1. **Proposta de Código:** O Agente Dev submete um novo módulo ou alteração de código ao OmegaCore .
2. **Provisionamento da Sandbox:** O OmegaCore orquestra a criação de uma nova Sandbox (e.g., container Docker ou VM efêmera) totalmente isolada e limpa.
3. **Injeção de Código e Dados:** O código proposto e dados de teste (incluindo *mocks* para APIs externas) são injetados na Sandbox.
4. **Auditoria Ofensiva:** O Agente Hacker é instruído pelo OmegaCore a iniciar sua bateria de testes e simulações de ataque dentro dessa Sandbox.
5. **Coleta de Resultados:** Logs de execução, relatórios de vulnerabilidades e resultados dos testes são coletados da Sandbox pelo OmegaCore .
6. **Destruição da Sandbox:** A Sandbox é imediatamente eliminada, garantindo que nenhum artefato malicioso ou estado indesejado persista 5 .

2.3. Prevenção de Erros Sistêmicos

- **Isolamento de Rede:** A Sandbox opera em uma rede virtual isolada, sem rotas para o ambiente de produção. Isso impede que um código comprometido na Sandbox tente acessar ou atacar o OmegaCore .
- **Limitação de Recursos (cgroups):** Mecanismos de controle de grupos (cgroups no Linux) são utilizados para limitar o uso de CPU, memória e I/O da Sandbox. Isso evita que um código em loop infinito ou um ataque de DoS dentro da Sandbox afete a performance do servidor host 6 .
- **Privilégios Mínimos:** Os processos dentro da Sandbox são executados com o menor conjunto de privilégios possível, reduzindo a superfície de ataque.

3. Protocolo de Comunicação: Orquestração DevSecOps

A comunicação entre os agentes e o OmegaCore é baseada em um **Modelo de Mensageria Assíncrona (Event-Driven)**, garantindo desacoplamento e resiliência. Os agentes não

interagem diretamente, mas sim através de um barramento de eventos orquestrado pelo OmegaCore 7 .

3.1. Fluxo de Comunicação Estruturado

1. **Agente Dev Submete Proposta:** O Agente Dev envia uma mensagem ao OmegaCore com o código-fonte de uma nova funcionalidade ou correção, acompanhada de metadados (versão, descrição, testes unitários).
2. **OmegaCore Orquestra Sandbox:** O OmegaCore recebe a proposta, provisiona uma Sandbox, injeta o código e os *mocks* necessários.
3. **OmegaCore Solicita Auditoria:** O OmegaCore envia uma mensagem ao Agente Hacker instruindo-o a iniciar a auditoria ofensiva no código recém-implantado na Sandbox.
4. **Agente Hacker Reporta Resultados:** O Agente Hacker executa seus testes e envia um relatório estruturado (JSON) de volta ao OmegaCore , detalhando vulnerabilidades encontradas, criticidade e evidências.
5. **OmegaCore Toma Decisão:** Com base no relatório do Agente Hacker e em outras políticas de segurança, o OmegaCore decide se o código pode ser promovido para integração ou se precisa de revisão.
6. **Feedback ao Agente Dev:** O OmegaCore envia uma mensagem ao Agente Dev com o resultado da auditoria, incluindo detalhes das vulnerabilidades para correção, se necessário.

3.2. Estrutura da Mensagem (JSON)

Todas as comunicações entre os agentes e o OmegaCore utilizam um formato JSON padronizado para garantir clareza, rastreabilidade e automação da análise. Exemplo de um relatório de vulnerabilidade do Agente Hacker :

JSON

```
{
  "origem": "agente_hacker",
  "destino": "omega_core",
  "transacao_id": "tx_devsecops_001",
  "timestamp": "2026-04-25T10:30:00Z",
  "tipo_mensagem": "RELATORIO_VULNERABILIDADE",
  "payload": {
    "codigo_versao": "v1.2.3",
    "status_auditoria": "FALHA",
    "vulnerabilidades": [
      {
        "id": "CVE-2026-001",
```

```

        "nome": "Injeção de Comando Potencial",
        "criticidade": "CRÍTICA",
        "descricao": "A função 'executar_comando' não sanitiza a entrada do usu
        "localizacao": "modulo_trading.py:linha_42",
        "evidencia_url": "s3://ravena-logs/sandbox-output-tx_devsecops_001.log"
        "recomendacao": "Implementar validação rigorosa de entrada e usar subpr
    }
],
"resumo_execucao_sandbox": {
    "duracao_segundos": 120,
    "cpu_max_percent": 85,
    "memoria_max_mb": 512
}
}
}

```

3.3. Hierarquia de Decisão e Veto de Segurança

- **Veto do Agente Hacker:** Se o Agente Hacker identificar e reportar uma vulnerabilidade de criticidade CRÍTICA ou ALTA, o OmegaCore **automaticamente bloqueia** a integração do código. A segurança tem precedência sobre a funcionalidade.
- **Arbitragem do OmegaCore:** O OmegaCore atua como o ponto central de orquestração e decisão. Ele aplica as políticas de segurança definidas pela SecurityLayer e Lockdown antes de permitir qualquer alteração no ambiente de produção, garantindo que o ciclo DevSecOps seja cumprido integralmente.

Conclusão

A introdução do Agente Hacker e a implementação de Sandboxes isoladas representam um avanço significativo na segurança e robustez da Ravena AI. Esta arquitetura conceitual da v3.3.0 estabelece um modelo DevSecOps onde a segurança é um componente intrínseco e contínuo do ciclo de desenvolvimento, e não uma etapa posterior. Ao isolar a execução de código e simular ataques em ambientes controlados, a Ravena pode evoluir rapidamente com a confiança de que sua integridade sistêmica está protegida contra falhas e ameaças

8 .

Referências

- [1] Nygard, M. (2018). Release It! Second Edition: Design and Deploy Production-Ready Software. The Pragmatic Bookshelf.
- [2] Microsoft. What is DevSecOps?. Disponível em:
- [3] OWASP Foundation. Vulnerability Scanning vs. Penetration Testing. Disponível em:

- [4] Docker Documentation. Isolate containers with a user namespace. Disponível em:
- [5] NIST Special Publication 800-190. Application Container Security Guide. Disponível em:
- [6] Linux Foundation. cgroups - control groups. Disponível em:
- [7] Martin Fowler. What do you mean by "Event-Driven"?. Disponível em:
- [8] DevSecOps Manifesto. Security as Code. Disponível em: